ASP.NET Web API

Tag 3 – Datenbanken, Logging und Testen

4-Tage-Kurs | Alexander Fischer, MSc

Wiederholung: Was wisst ihr noch?

Wiederholung: Was wisst ihr noch?

1. Welche drei Lifetimes gibt es, und wann nehmt ihr welche?

Wiederholung: Was wisst ihr noch?

1. Welche drei Lifetimes gibt es, und wann nehmt ihr welche?
2. Welche Datei gewinnt: `appsettings.json` oder `appsettings.Development.json`?

Wiederholung: Was wisst ihr noch?

1. Welche drei Lifetimes gibt es, und wann nehmt ihr welche?
2. Welche Datei gewinnt: `appsettings.json` oder `appsettings.Development.json`?
3. Wie fordert ein Controller typsichere Konfiguration an?

Wiederholung: Was wisst ihr noch?

1. Welche drei Lifetimes gibt es, und wann nehmt ihr welche?
2. Welche Datei gewinnt: `appsettings.json` oder `appsettings.Development.json`?
3. Wie fordert ein Controller typsichere Konfiguration an?
4. Was passiert bei ungültigen Daten, bevor die Aktion läuft?

Wiederholung: Was wisst ihr noch?

1. Welche drei Lifetimes gibt es, und wann nehmt ihr welche?
2. Welche Datei gewinnt: `appsettings.json` oder `appsettings.Development.json`?
3. Wie fordert ein Controller typsichere Konfiguration an?
4. Was passiert bei ungültigen Daten, bevor die Aktion läuft?
5. Warum trennt man Entity und DTO?

Entity Framework Core

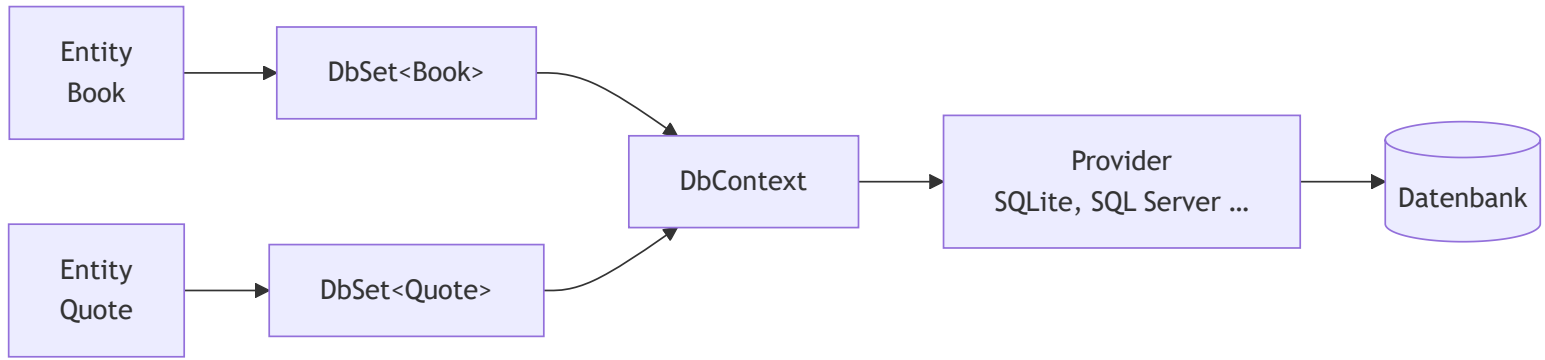
Was ist ein ORM?




```
var books = await _db.Books.Where(b ⇒ b.Year > 2000).ToListAsync();
// SELECT ... FROM Books WHERE Year > 2000
```

- Kein SQL-String im Code, Typprüfung durch den Compiler
- Datenbankanbieter austauschbar

Die Bausteine von EF Core



Code First vs. Database First

	Code First	Database First
Start	C#-Klassen	bestehende Datenbank
Ergebnis	Datenbank wird erzeugt	Klassen werden erzeugt
Werkzeug	`migrations add`	`dbcontext scaffold`
Typisch für	neue Projekte	bestehende Systeme

Warum SQLite im Kurs?

- Datenbank in **einer einzigen Datei**
- Kein Server, keine Installation, kein Passwort
- Code funktioniert fast unverändert mit SQL Server oder PostgreSQL
- Nur Provider und Connection String ändern sich

Pakete und Werkzeug

```
dotnet add package Microsoft.EntityFrameworkCore.Sqlite
dotnet add package Microsoft.EntityFrameworkCore.Design
dotnet tool install --global dotnet-ef
```

> **Paketversion = .NET-Version!** Unter .NET 8: `--version 8.*` anhängen.

Entity und DbContext

```
public class Book
{
    public int Id { get; set; }              // Primärschlüssel per Konvention
    public string Title { get; set; } = "";
    public string Author { get; set; } = "";
}

public class LibraryContext : DbContext
{
    public LibraryContext(DbContextOptions<LibraryContext> options)
        : base(options) { }

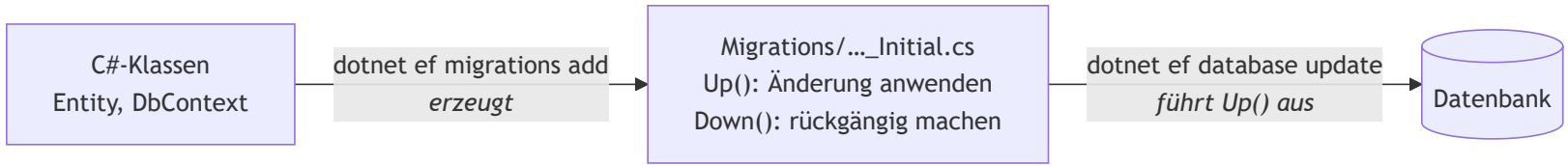
    public DbSet<Book> Books ⇒ Set<Book>();
}
```

DbContext registrieren

```
// Program.cs – registriert als Scoped
builder.Services.AddDbContext<LibraryContext>(o ⇒
    o.UseSqlite(builder.Configuration.GetConnectionString("Library")));
```

```
// appsettings.json
"ConnectionStrings": {
  "Library": "Data Source=library.db"
}
```

Migrationen



- `migrations add` erzeugt **nur die C#-Datei** – die Datenbank bleibt unverändert
- Versionsverwaltung für das Schema – gehört in Git
- Tabelle `__EFMigrationsHistory` merkt sich angewendete Migrationen

Code First – Ablauf auf einen Blick

1. Pakete installieren
2. Entity schreiben
3. DbContext schreiben
4. Context in `Program.cs` registrieren
5. Connection String in `appsettings.json`
6. `dotnet ef migrations add Initial`
7. `dotnet ef database update`

Übung 10 – Code First mit SQLite

20 Minuten

1. Entity `Book` mit Id, Title, Author
2. `LibraryContext` mit `DbSet<Book>`
3. Mit SQLite registrieren, Connection String in appsettings
4. Erste Migration erstellen
5. Datenbank anlegen lassen

Ergebnis: Ordner `Migrations/` und Datei `library.db`

Lösung Übung 10

```
dotnet add package Microsoft.EntityFrameworkCore.Sqlite
dotnet add package Microsoft.EntityFrameworkCore.Design
dotnet tool install --global dotnet-ef

dotnet ef migrations add Initial
dotnet ef database update
```

> „Unable to create a DbContext“? Registrierung prüfen und ob das Projekt baut.

EF 6.5 vs. EF Core

	Entity Framework 6.5	EF Core
Herkunft	seit 2008, Version 6 seit 2013	2016 komplett neu geschrieben
Läuft auf	.NET Framework, ab 6.3 auch .NET	.NET (8, 10 …)
Stand	nur Wartung, keine neuen Features	jedes Jahr neue Version mit .NET
Datenbanken	vor allem SQL Server	SQL Server, SQLite, PostgreSQL, MySQL, Cosmos DB …
Database First	EDMX-Designer in Visual Studio	`dotnet ef dbcontext scaffold`
Lazy Loading	Standard (`virtual` -Navigationen)	aus, nur per Zusatzpaket

EF 6.5 und EF Core im Code

EF 6.5

```
using System.Data.Entity;

public class LibraryContext : DbContext
{
    public LibraryContext()
        : base("name=Library") { }
        // aus web.config / app.config

    public DbSet<Book> Books { get; set; }
}
```

```
# Package Manager Console
Add-Migration Initial
Update-Database
```

EF Core

```
using Microsoft.EntityFrameworkCore;

public class LibraryContext : DbContext
{
    public LibraryContext(
        DbContextOptions<LibraryContext> o)
        : base(o) { }   // per DI

    public DbSet<Book> Books => Set<Book>();
}
```

```
dotnet ef migrations add Initial
dotnet ef database update
```

Was tun mit EF6-Anwendungen?

- **Neue Projekte:** immer EF Core
- **Bestehende .NET-Framework-Anwendung:** EF6 läuft stabil weiter
- **Umstieg auf .NET:** EF 6.5 läuft auch unter .NET – erst die Plattform wechseln, dann das ORM
- **EF6 → EF Core ist eine Portierung, kein Update:**
  - andere Namespaces und Konfiguration
  - EDMX gibt es nicht mehr – neu scaffolden
  - Verhalten prüfen: Lazy Loading, Abfragen, Löschen verknüpfter Daten

> LINQ-Abfragen sehen fast gleich aus – Tests vor dem Umstieg sind Pflicht.

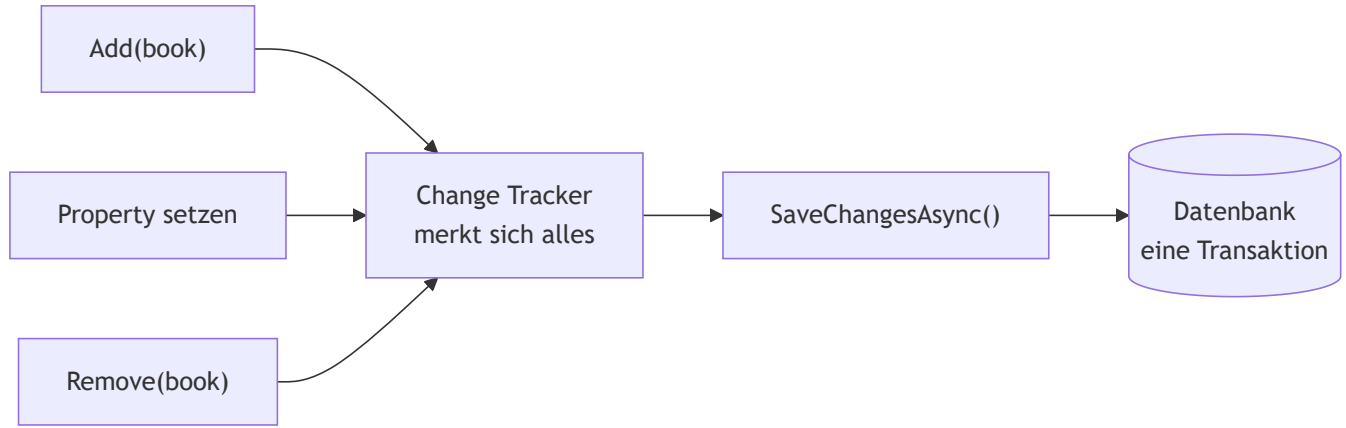
CRUD mit EF Core

Lesen mit LINQ

Methode	Ergebnis
`ToListAsync()`	alle Einträge
`FindAsync(id)`	über Primärschlüssel oder `null`
`FirstOrDefaultAsync(b ⇒ …)`	erster Treffer oder `null`
`Where(…)`, `OrderBy(…)`	filtern, sortieren
`Skip(20).Take(10)`	blättern

> Die Abfrage läuft erst bei `ToListAsync` & Co. – vorher wird nur SQL gebaut.

Schreiben mit dem Change Tracker



Nach `SaveChangesAsync` steht die neue Id automatisch im Objekt.

Warum async?

- Während die Datenbank arbeitet, wartet der Thread
- Mit `await` wird er frei für andere Anfragen
- Mehr gleichzeitige Anfragen mit derselben Hardware

```
public async Task<ActionResult<Book>> Get(int id)
{
    var book = await _db.Books.FindAsync(id);
    ...
}
```

> Nie `.Result` oder `.Wait()` – blockiert und kann zu Deadlocks führen.

Der BooksController (1/2)

```
private readonly LibraryContext _db;
public BooksController(LibraryContext db) => _db = db;

[HttpGet]
public async Task<ActionResult<IEnumerable<Book>>> GetAll()
    => Ok(await _db.Books.ToListAsync());

[HttpGet("{id:int}")]
public async Task<ActionResult<Book>> Get(int id)
{
    var book = await _db.Books.FindAsync(id);
    if (book is null) return NotFound();
    return book;
}
```

Der BooksController (2/2)

```
[HttpPost]
public async Task<ActionResult<Book>> Create(Book book)
{
    _db.Books.Add(book);
    await _db.SaveChangesAsync();
    return CreatedAtAction(nameof(Get), new { id = book.Id }, book);
}
```

PUT und DELETE folgen demselben Muster – in der Übung.

Übung 11 – CRUD mit EF Core

20 Minuten

1. `BooksController` mit GET (alle, einzeln) und POST
2. Ergänze **PUT** und **DELETE** – jeweils 404, wenn nicht vorhanden
3. Feld `Year` zu `Book` hinzufügen
4. Zweite Migration erstellen und anwenden

Lösung Übung 11

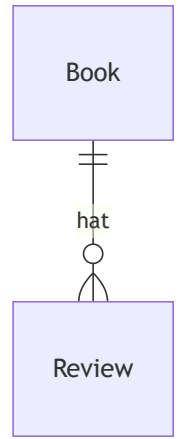
```
[HttpPut("{id:int}")]
public async Task<IActionResult>
    Update(int id, Book input)
{
    var book = await _db.Books
        .FindAsync(id);
    if (book is null) return NotFound();
    book.Title = input.Title;
    book.Author = input.Author;
    await _db.SaveChangesAsync();
    return NoContent();
}
```

```
[HttpDelete("{id:int}")]
public async Task<IActionResult>
    Delete(int id)
{
    var book = await _db.Books
        .FindAsync(id);
    if (book is null) return NotFound();
    _db.Books.Remove(book);
    await _db.SaveChangesAsync();
    return NoContent();
}
```

```
dotnet ef migrations add AddYear
dotnet ef database update
```

Beziehungen zwischen Entities

Beziehungsarten



Art	Beispiel	In C#
1:n	Buch – Rezensionen	Liste auf der einen, Referenz auf der anderen Seite
n:m	Buch – Kategorien	Listen auf beiden Seiten, Zwischentabelle macht EF Core
1:1	Buch – Cover	Referenz auf beiden Seiten

Navigation Properties

```
public class Book
{
    // Id, Title, Author, Year …
    public List<Review> Reviews { get; set; } = new();
}

public class Review
{
    public int Id { get; set; }
    public int Stars { get; set; }
    public string Comment { get; set; } = "";
    public int BookId { get; set; }      // Fremdschlüssel per Konvention
    public Book? Book { get; set; }      // Navigation zurück
}
```

> `BookId` ist `int`, also Pflicht → **Cascade Delete**: Buch weg, Rezensionen weg.

Drei Arten, Beziehungen zu laden

EF Core lädt Navigationen **nicht automatisch** – ohne eine dieser Arten bleibt `book.Reviews` leer.

Art	Wann wird geladen?	Wie
Eager	sofort, mit der Hauptabfrage	`Include(b ⇒ b.Reviews)`
Explicit	später, wenn der Code es anfordert	`Entry(book).Collection(…).LoadAsync()`
Lazy	automatisch beim ersten Zugriff	Paket `Microsoft.EntityFrameworkCore.Proxies`

> Faustregel für APIs: **Eager** oder Projektion. Explicit, wenn es nur manchmal gebraucht wird. Lazy meiden.

Eager und Explicit Loading

Eager – ein SQL mit JOIN

```
var book = await _db.Books
    .Include(b => b.Reviews)
    .FirstOrDefaultAsync(b => b.Id == id);

// gefiltert (ab EF Core 5)
.Include(b => b.Reviews
    .Where(r => r.Stars >= 4))

// weiter verschachtelt
.Include(b => b.Reviews)
    .ThenInclude(r => r.Author)
```

Explicit – zweites SQL, nur bei Bedarf

```
var book = await _db.Books.FindAsync(id);

if (book is not null && withReviews)
{
    await _db.Entry(book)
        .Collection(b => b.Reviews)
        .LoadAsync();
}

// Einzelreferenz statt Liste
await _db.Entry(review)
    .Reference(r => r.Book)
    .LoadAsync();
```

Lazy Loading mit Proxies

```
dotnet add package Microsoft.EntityFrameworkCore.Proxies
```

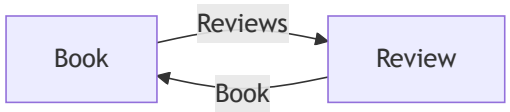
```
o.UseSqlite(connectionString).UseLazyLoadingProxies();

public virtual List<Review> Reviews { get; set; } = new();   // virtual ist Pflicht
```

```
var books = await _db.Books.ToListAsync();      // 1 Abfrage
foreach (var b in books)
    Console.WriteLine(b.Reviews.Count);         // + 1 Abfrage pro Buch → N+1
```

- Lädt **synchron** und unsichtbar – auch beim JSON-Serialisieren
- Alle Navigationen aller Entities müssen `virtual` sein

Zyklen bei der Serialisierung




```
JsonException: A possible object cycle was detected.
```

Ausweg	Bewertung
`[JsonIgnore]` an `Review.Book`	schnell, Entity bestimmt das JSON
`ReferenceHandler.IgnoreCycles`	versteckt das Problem
Projektion mit `Select`	empfohlen

Projektion mit Select

```
var book = await _db.Books
    .Where(b => b.Id == id)
    .Select(b => new
    {
        b.Id,
        b.Title,
        AverageStars = b.Reviews.Average(r => (double?)r.Stars),
        Reviews = b.Reviews.Select(r => new { r.Stars, r.Comment }).ToList()
    })
    .FirstOrDefaultAsync();
```

- Ein SQL, nur die benötigten Spalten, `AVG` rechnet die Datenbank
- Kein `Include` nötig, keine Zyklen

> `(double?)`: Ohne Rezensionen liefert SQL NULL – ohne Cast gibt es eine Exception.

Beziehungen – auf einen Blick

1. Navigation Properties auf beiden Seiten, Fremdschlüssel `BookId`
2. `migrations add` – Fremdschlüssel, Index und Cascade Delete entstehen automatisch
3. Laden: Eager (`Include`), Explicit (`Entry…LoadAsync`), Lazy (Proxies) – für APIs oft besser: **Projektion**
4. Entities mit Navigationen nie direkt als JSON zurückgeben
5. Eingaben über ein DTO statt über die Entity

Übung 11b – Rezensionen auswerten

10 Minuten

Voraussetzung: `Review`, Migration `AddReviews` und `POST api/books/{id}/reviews` aus dem Live-Coding

1. `[Range(1, 5)]` an `Stars` im `ReviewDto` – 0 Sterne ergibt 400
2. Neuer Endpoint `GET api/books/ratings`: pro Buch Titel, Anzahl Rezensionen, Durchschnitt
3. Optional: absteigend nach Durchschnitt sortieren

Schaut euch das erzeugte SQL in der Konsole an: Wie viele Statements sind es?

Lösung Übung 11b

```
[HttpGet("ratings")]
public async Task<IActionResult> GetRatings()
    => Ok(await _db.Books
        .Select(b => new
        {
            b.Title,
            ReviewCount = b.Reviews.Count,
            AverageStars = b.Reviews.Average(r => (double?)r.Stars)
        })
        .OrderByDescending(x => x.AverageStars)
        .ToListAsync());
```

Ein Statement mit Unterabfragen für `COUNT` und `AVG` – keine Rezension wandert übers Netz.

Database First

```
dotnet ef dbcontext scaffold "Data Source=shop.db" \
  Microsoft.EntityFrameworkCore.Sqlite -o Models/Shop
```

- Erzeugt Entities und Context aus einer bestehenden Datenbank
- Klassen sind `partial` – in eigener Datei erweiterbar
- `OnConfiguring` mit Connection String entfernen, in `Program.cs` registrieren

EF Core mit MSSQL

```
dotnet add package Microsoft.EntityFrameworkCore.SqlServer
```

```
o.UseSqlServer(builder.Configuration.GetConnectionString("Library"))
```

```
Server=(localdb)\MSSQLLocalDB;Database=Library;
Trusted_Connection=True;TrustServerCertificate=True
```

> Migrationen sind providerabhängig – nach dem Wechsel neu erstellen.

Logging

ILogger<T>

```
public class BooksController : ControllerBase
{
    private readonly ILogger<BooksController> _logger;

    public BooksController(LibraryContext db, ILogger<BooksController> logger)
    {
        _db = db;
        _logger = logger;
    }
}
```

- Per DI angefordert, `T` wird zur **Kategorie**
- Standardziele: Konsole, Debug-Fenster

Log-Level

Level	Wofür	Beispiel
`Trace`, `Debug`	Details für die Entwicklung	Zwischenwerte
`Information`	normale Abläufe	Buch angelegt
`Warning`	Ungewöhnliches	Buch nicht gefunden
`Error`	Vorgang fehlgeschlagen	Speichern fehlgeschlagen
`Critical`	Anwendung in Gefahr	Datenbank nicht erreichbar

Strukturiertes Logging

```
// Falsch: fertiger Text, Werte gehen verloren
_logger.LogInformation($"Buch {book.Id} angelegt: {book.Title}");

// Richtig: Text + Felder BookId und Title
_logger.LogInformation("Buch {BookId} angelegt: {Title}", book.Id, book.Title);
```

Immer Platzhalter, nie Interpolation. Nur so könnt ihr später nach `BookId = 3` filtern.

Log-Level konfigurieren

```
"Logging": {
  "LogLevel": {
    "Default": "Information",
    "Microsoft.AspNetCore": "Warning",
    "Microsoft.EntityFrameworkCore.Database.Command": "Information"
  }
}
```

- Pro Kategorie, gilt für alle Kategorien mit diesem Präfix
- Letzte Zeile: zeigt jedes SQL-Statement von EF Core

Übung 12 – Logging

15 Minuten

Im `BooksController`:

- Anlegen und Löschen als **Information** loggen
- Nicht gefundenes Buch als **Warning** loggen
- Log-Level so einstellen, dass die SQL-Befehle erscheinen

Lösung Übung 12

```
// in Create, nach SaveChangesAsync
_logger.LogInformation("Buch {BookId} angelegt: {Title}", book.Id, book.Title);

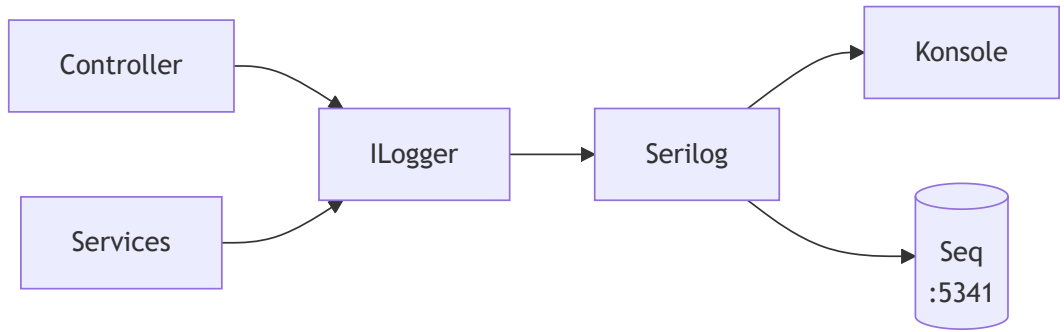
// in Delete, nach SaveChangesAsync
_logger.LogInformation("Buch {BookId} gelöscht", id);

// in Get, wenn book null ist
_logger.LogWarning("Buch {BookId} nicht gefunden", id);
```

Serilog und Seq

- **Serilog** hängt sich in das bestehende Logging ein – `ILogger` bleibt
- **Sinks**: Konsole, Datei mit Rotation, Datenbank, Cloud, Seq
- **Seq**: zentrale Oberfläche zum Durchsuchen strukturierter Logs
- Seq läuft am einfachsten im Docker-Container

Logging-Architektur mit Serilog und Seq



Serilog einrichten

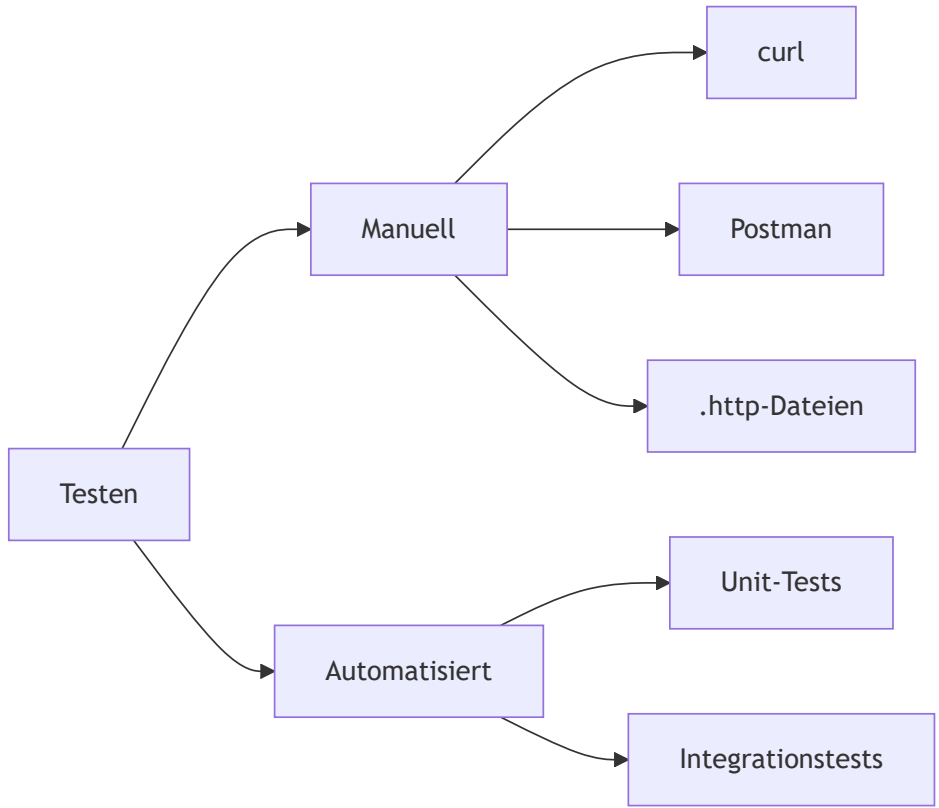
```
docker run -d --name seq -e ACCEPT_EULA=Y \
  -e SEQ_FIRSTRUN_NOAUTHENTICATION=true -p 5341:80 datalust/seq
dotnet add package Serilog.AspNetCore
dotnet add package Serilog.Sinks.Seq
```

```
builder.Host.UseSerilog((ctx, cfg) ⇒ cfg
    .ReadFrom.Configuration(ctx.Configuration)
    .WriteTo.Console()
    .WriteTo.Seq("http://localhost:5341"));
```

Oberfläche: `http://localhost:5341` · Filter z. B. `BookId = 3`

Testen der REST-API

Manuell oder automatisiert?



curl für Fortgeschrittene

```
curl -k -i -X POST https://localhost:<port>/api/books \
  -H "Content-Type: application/json" \
  -d '{ "title": "Der Hobbit", "author": "J.R.R. Tolkien" }'
```

Teil	Bedeutung
`-i`	Status und Header anzeigen
`-X POST`	Methode
`-H`	Content-Type setzen
`-d`	JSON-Body

Postman

- Grafisches Werkzeug für API-Tests
- **Collections** – Anfragen speichern und gruppieren
- **Variablen** – Basis-URL, Tokens
- **Tests** – Status Codes automatisch prüfen
- **Import** – aus dem OpenAPI-Dokument

```
pm.test("Status 201", () ⇒ pm.response.to.have.status(201));
```

.http-Dateien

```
@host = https://localhost:5001

GET {{host}}/api/books

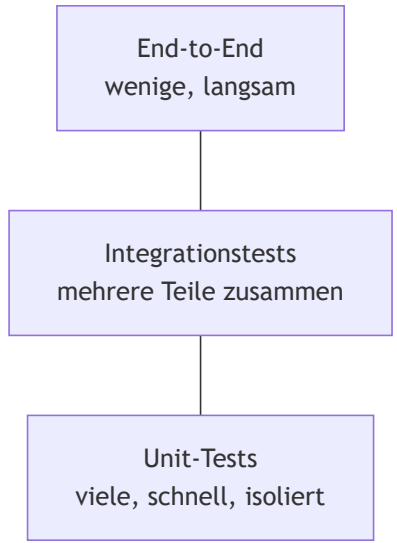
###
POST {{host}}/api/books
Content-Type: application/json

{ "title": "Der Hobbit", "author": "J.R.R. Tolkien", "year": 1937 }

###
DELETE {{host}}/api/books/1
```

Visual Studio und Rider direkt · VS Code mit „REST Client“ · liegt im Projekt und in Git

Automatisierte Tests – die Testpyramide



Testframeworks in .NET

Framework	Test-Attribut	Bemerkung
xUnit	`[Fact]`, `[Theory]`	verbreitetstes, im Kurs
NUnit	`[Test]`	lange etabliert
MSTest	`[TestMethod]`	von Microsoft

Ein xUnit-Test

```
public class GreetingServiceTests
{
    [Fact]
    public void Greet_ContainsName()
    {
        var service = new GreetingService();          // Arrange
        var result = service.Greet("Anna");           // Act
        Assert.Equal("Hallo Anna", result);           // Assert
    }

    [Theory]
    [InlineData("Anna")]
    [InlineData("Ben")]
    public void Greet_StartsWithHallo(string name)
        => Assert.StartsWith("Hallo", new GreetingService().Greet(name));
}
```

Testprojekt anlegen

```
cd ..
dotnet new xunit -n KursApi.Tests
dotnet add KursApi.Tests reference KursApi
dotnet sln add KursApi.Tests
cd KursApi.Tests
dotnet test
```

```
kurs/tag3/
├── Tag3.slnx
├── KursApi/          API-Projekt
└── KursApi.Tests/    Testprojekt – daneben, nicht darin
```

Übung 13 – .http-Datei und erster Unit-Test

20 Minuten

1. `.http`-Datei mit allen Book-Aufrufen anlegen
2. xUnit-Testprojekt neben dem API-Projekt erstellen und mit `dotnet sln add` in die Solution aufnehmen
3. Projektreferenz auf `KursApi` setzen
4. `GreetingService` aus Übung 5 testen
5. `dotnet test` – grün?

Lösung Übung 13

```
public class GreetingServiceTests
{
    [Fact]
    public void Greet_ContainsName()
    {
        var service = new GreetingService();
        Assert.Equal("Hallo Anna", service.Greet("Anna"));
    }

    [Fact]
    public void TwoInstances_HaveDifferentIds()
    {
        Assert.NotEqual(new GreetingService().InstanceId,
                        new GreetingService().InstanceId);
    }
}
```

Challenge 3 – Zitate-API mit Datenbank

35 Minuten

Verwendet: Tag 1 und 2, dazu EF Core Code First, Migrationen, Logging, `.http`-Datei, xUnit, NuGet

> Gestern war der QuoteService ein Singleton. Der DbContext ist Scoped. Was bedeutet das?

Challenge 3 – Anforderungen

- `Quote` wird eine Entity im `LibraryContext`, mit Migration
- `QuoteService` arbeitet mit EF Core – **welche Lifetime jetzt?**
- Loggen: Anlegen, Löschen, Maximum erreicht
- `.http`-Datei mit allen Zitat-Aufrufen
- Test: Ist das Maximum erreicht, gibt `AddAsync` `null` zurück

Tipp: Paket `Microsoft.EntityFrameworkCore.InMemory` im Testprojekt · `Options.Create` · `NullLogger`

Lösung Challenge 3 (1/4)

```
// LibraryContext
public DbSet<Quote> Quotes => Set<Quote>();

public interface IQuoteService
{
    Task<List<Quote>> GetAllAsync();
    Task<Quote?> GetAsync(int id);
    Task<Quote?> AddAsync(Quote quote);
    Task<bool> DeleteAsync(int id);
}
```

```
dotnet ef migrations add AddQuotes
dotnet ef database update
```

Lösung Challenge 3 (2/4)

```
public async Task<Quote?> AddAsync(Quote quote)
{
    if (await _db.Quotes.CountAsync() ⩾ _options.MaxQuotes)
    {
        _logger.LogWarning("Maximum von {Max} Zitaten erreicht", _options.MaxQuotes);
        return null;
    }
    quote.Id = 0;   // die Datenbank vergibt die Id
    _db.Quotes.Add(quote);
    await _db.SaveChangesAsync();
    _logger.LogInformation("Zitat {QuoteId} angelegt", quote.Id);
    return quote;
}
```

Konstruktor bekommt `LibraryContext`, `IOptions<QuoteOptions>`, `ILogger<QuoteService>`

Lösung Challenge 3 (3/4)

```
public async Task<bool> DeleteAsync(int id)
{
    var quote = await _db.Quotes.FindAsync(id);
    if (quote is null) return false;
    _db.Quotes.Remove(quote);
    await _db.SaveChangesAsync();
    _logger.LogInformation("Zitat {QuoteId} gelöscht", id);
    return true;
}
```

```
// Program.cs – Scoped, weil der DbContext Scoped ist
builder.Services.AddScoped<IQuoteService, QuoteService>();
```

Lösung Challenge 3 (4/4)

```
[Fact]
public async Task AddAsync_ReturnsNull_WhenMaximumReached()
{
    var dbOptions = new DbContextOptionsBuilder<LibraryContext>()
        .UseInMemoryDatabase(Guid.NewGuid().ToString())
        .Options;
    using var db = new LibraryContext(dbOptions);
    var service = new QuoteService(db,
        Options.Create(new QuoteOptions { MaxQuotes = 1 }),
        NullLogger<QuoteService>.Instance);

    Assert.NotNull(await service.AddAsync(new Quote { Text = "Eins" }));
    Assert.Null(await service.AddAsync(new Quote { Text = "Zwei" }));
}
```

Rückblick Tag 3

Rückblick Tag 3

- EF Core: Code First, Migrationen, asynchrones CRUD

Rückblick Tag 3

- EF Core: Code First, Migrationen, asynchrones CRUD
- Beziehungen: Eager, Explicit und Lazy Loading, Projektion mit `Select`

Rückblick Tag 3

- EF Core: Code First, Migrationen, asynchrones CRUD
- Beziehungen: Eager, Explicit und Lazy Loading, Projektion mit `Select`
- Database First, Wechsel zu MSSQL, EF 6.5 vs. EF Core

Rückblick Tag 3

- EF Core: Code First, Migrationen, asynchrones CRUD
- Beziehungen: Eager, Explicit und Lazy Loading, Projektion mit `Select`
- Database First, Wechsel zu MSSQL, EF 6.5 vs. EF Core
- Strukturiertes Logging, Serilog und Seq

Rückblick Tag 3

- EF Core: Code First, Migrationen, asynchrones CRUD
- Beziehungen: Eager, Explicit und Lazy Loading, Projektion mit `Select`
- Database First, Wechsel zu MSSQL, EF 6.5 vs. EF Core
- Strukturiertes Logging, Serilog und Seq
- Testen mit curl, Postman, `.http`-Dateien und xUnit

Rückblick Tag 3

- EF Core: Code First, Migrationen, asynchrones CRUD
- Beziehungen: Eager, Explicit und Lazy Loading, Projektion mit `Select`
- Database First, Wechsel zu MSSQL, EF 6.5 vs. EF Core
- Strukturiertes Logging, Serilog und Seq
- Testen mit curl, Postman, `.http`-Dateien und xUnit
- Die Zitate-API speichert in einer Datenbank

Ausblick Tag 4

Fehlerbehandlung · JWT und Rollen · Caching · Docker · Health Checks



Mühlehner & Tavolato GmbH

SCHULUNG · BERATUNG · ENTWICKLUNG

AF

Alexander Fischer

af@mit.at

Kurse

offen und als Firmenschulung

Individualschulungen

zugeschnitten auf euer Team

Consulting

Beratung für eure Projekte

Auftragsentwicklung

wir bauen eure Software